\WOOLF/

Applied Software Project Report

By

Aravind Karteek Rayavarapu

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

SCALER

August, 2026

Scaler Mentee Email ID : aravindkarteek@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 11/08/2026

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

……

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 25/03/2026 to 29/04/2026 is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Aravind Karteek Rayavarapu

<Signature of the Candidate> **Date: 11/08/2026**

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my thesis supervisor, Naman Bhalla, and the entire faculty and mentor team at Scaler Neovarsity, for their invaluable guidance, timely feedback, and constant encouragement throughout the duration of this project. Their insights on real-world software engineering practices shaped not only this report but also the way I approach building and reasoning about production systems. I am equally grateful to the broader Scaler and Woolf community for creating a rigorous yet supportive environment that pushed me to grow as an engineer. Finally, I want to thank my family for their unwavering patience, encouragement, and support throughout this program — for the late nights and weekends spent studying and building, and for always believing in me even when the path was demanding. This achievement would not have been possible without their constant motivation and sacrifice.

Table of Contents

Applied Software Project

Abstract

Online ticket booking for movies is deceptively hard to get right: many users can try to reserve the same seat within milliseconds of each other, payments must be confirmed asynchronously without ever double-charging or double-booking a customer, and the platform must serve three kinds of users — moviegoers, theatre owners, and administrators — each with different access and write privileges. **TicketVerse** is a full-stack, production-representative movie ticket booking platform built to solve this problem, developed as an Applied Software Project using the MERN stack (MongoDB, Express, React, Node.js) inside a TypeScript monorepo.

The purpose was to design and ship a working, secure, independently deployed system covering the full customer journey — browsing movies and showtimes, selecting seats on a live seat map, holding seats safely under concurrent demand, paying through a real payment gateway, and receiving asynchronous confirmation — alongside role-based dashboards for theatre owners (managing screens and showtimes) and administrators (approving theatre-owner requests, curating the movie catalogue, and auditing bookings).

The methods applied include a Domain-Driven Design backend with clearly separated domain, application, infrastructure, and HTTP layers; Redis-based atomic seat holds (SETNX with a time-to-live) to prevent double-booking without locking the database; Stripe PaymentIntents and signature-verified webhooks for asynchronous, PCI-DSS-aligned payment confirmation; custom bcrypt password hashing with rotating JWT access/refresh tokens in httpOnly cookies; and a shared Zod schema package validating data identically on both backend and frontend.

The result is a fully tested (26 automated tests), fully deployed system — backend on Render, frontend on Netlify, database on MongoDB Atlas, cache/lock store on Upstash Redis — showing that the seat-hold and webhook-confirmation patterns used by real-world ticketing platforms can be reproduced correctly, securely, and economically on free-tier infrastructure.

Project Description

Overview

TicketVerse is a movie ticket booking platform that lets a moviegoer search for a movie, browse showtimes across theatres in a city, pick seats from a live seat map, hold those seats while paying, and receive a confirmed booking once payment succeeds. Alongside the customer-facing flow, the platform supports two additional roles: a **theatre owner**, who can list their theatres, define each screen's seat layout, and schedule showtimes against the platform's shared movie catalogue; and an **administrator**, who curates the movie catalogue, approves or rejects theatre-owner applications, and has read-only oversight of all users, theatres, and bookings for support and auditing purposes.

The project was built as a TypeScript monorepo (apps/web, apps/api, packages/schemas, packages/config) managed with npm workspaces and Turborepo, so the frontend and backend can share one source of truth for request/response validation instead of the two ends of the API silently drifting apart over time, which is a common real-world source of production bugs.

Objectives

1. Build a genuinely **concurrency-safe** seat-booking mechanism — the core technical challenge of any ticketing system — rather than a naive implementation that would allow two customers to pay for the same seat.
2. Implement a **complete, asynchronous payment flow** using a real payment gateway (Stripe), including signature-verified webhook confirmation, mirroring how production ticketing and e-commerce systems actually confirm payment rather than trusting the client's browser.
3. Design a **role-based access control (RBAC)** system from first principles (custom bcrypt + JWT implementation, not a third-party auth provider) so that the reasoning behind password hashing, token issuance, and token revocation is fully owned and explainable rather than hidden behind a vendor SDK.
4. Apply **Domain-Driven Design (DDD)** on the backend so that business rules (e.g., "a seat cannot be held twice", "only an admin can create a movie") live in a framework-independent domain layer, separated from Express routing and MongoDB persistence concerns.

5. **Deploy the system live** end-to-end (not just describe a hypothetical deployment) so that the operational concerns discussed in this report — CORS, cross-site cookies, environment configuration, managed database/cache provisioning — are grounded in a system that was actually exercised in production-like conditions.

Problem Statement

Most introductory full-stack tutorials build a CRUD (create/read/update/delete) application against a single resource with no contention between users. That shape teaches routing, forms, and database access, but it deliberately avoids the two problems that make ticketing, booking, and inventory systems hard in practice: **concurrency** (what happens when two people try to claim the same limited resource at the same instant) and **asynchronous settlement** (what happens when the system that confirms a transaction — a card network, a bank, a payment processor — is a separate, untrusted, and sometimes slow third party that the client cannot be trusted to report on honestly). TicketVerse was deliberately scoped around a domain — cinema seat booking — where both problems occur naturally and cannot be avoided or faked: a specific seat in a specific show is a genuinely scarce, non-substitutable resource, and Stripe's PaymentIntent/webhook model forces an asynchronous, server-to-server confirmation step that cannot be shortcut by trusting a success response in the browser. The problem, stated precisely, is: *given a fixed, finite set of seats for a scheduled show, guarantee that no seat is ever sold to more than one paying customer, while still allowing many customers to browse and attempt to book concurrently, and while the actual payment confirmation arrives asynchronously and out-of-band from the booking request itself.*

Scope and Out-of-Scope

To keep the project achievable within the time available while still exercising every technically interesting problem above, the following scope boundaries were set explicitly at the start of the project:

In scope: - Seat-level booking for a single-city, single-currency deployment (no multi-region pricing or tax logic). - Three roles — user, theatre owner, administrator — with role-based authorization enforced on the backend, not just hidden in the UI. - A single payment gateway integration (Stripe), fully wired end-to-end including webhook confirmation. - Automated backend and frontend tests covering the core concurrency-sensitive and security-sensitive code paths. - A real, live deployment across managed cloud services, rather than a purely local demo.

Out of scope (explicitly deferred, see also the Limitations section in the Conclusion chapter): - Multiple payment gateways or region-specific payment methods (e.g., UPI, wallets). - Seat-level dynamic pricing, discounts, or promotional codes. - Forgot-password / email-verification flows (the identity module supports signup, login, refresh, and logout only). - A notifications system (email/SMS booking confirmations) — bookings are confirmed synchronously in the API response and visible in the user's dashboard instead. - Horizontal scaling concerns beyond what a single Render web service instance and a single Upstash Redis instance provide; the system is designed to be *correct* under concurrency, not to be load-tested at a specific throughput target.

Stakeholders

Although this is an individually built academic project rather than a commercial product with real stakeholders, the system was designed around three internally consistent stakeholder personas so that the requirement-gathering and access-control decisions in later chapters have a concrete audience to be evaluated against:

- **The moviegoer** — wants to find a show, see an accurate, live seat map (not a stale one that leads to a failed booking after payment), and get a fast, unambiguous confirmation.
- **The theatre owner** — wants to list their own theatres and screens without needing platform-administrator involvement for routine operations (adding a screen, scheduling a show), but is willing to accept a one-time administrator approval step before being granted that capability, mirroring how real marketplace platforms (e.g., ride-sharing driver onboarding, e-commerce seller onboarding) vet new supply-side participants before granting them write access to shared inventory.
- **The platform administrator** — wants oversight (read access to all users, theatres, and bookings) and gatekeeping power (approving theatre-owner requests, curating the movie catalogue) without needing to perform day-to-day theatre operations themselves.

Relevance

The seat-hold-then-confirm pattern used in this project is the same pattern used by real airline, train, and event-ticketing platforms: a resource (a seat) is provisionally locked for a short window while the customer completes payment, and is only durably committed once an external payment provider confirms the charge asynchronously. Getting this pattern right — and getting it *wrong* in an obviously incorrect but tempting way (e.g., checking availability and booking in two separate, non-atomic steps) — is one of the most instructive concurrency problems a full-stack engineer

can work through, because the failure mode (double-booking a seat, or worse, double-charging a customer for a seat that was already released) is highly visible and consequential in the real world. The role-based dashboards additionally mirror a very common real-world SaaS shape: a public-facing consumer product with a separate, more privileged operator/admin surface layered on the same data.

Development Process

The project was implemented in six sequential phases — monorepo bootstrap, shared schema authoring, backend module-by-module implementation, frontend feature-by-feature implementation, integration wiring, and automated testing/documentation — followed by a seventh live-deployment phase once the application was feature-complete and tested. Each phase produced one or more incremental, working Git commits, so the commit history itself traces the system's evolution from an empty repository to a fully deployed application.

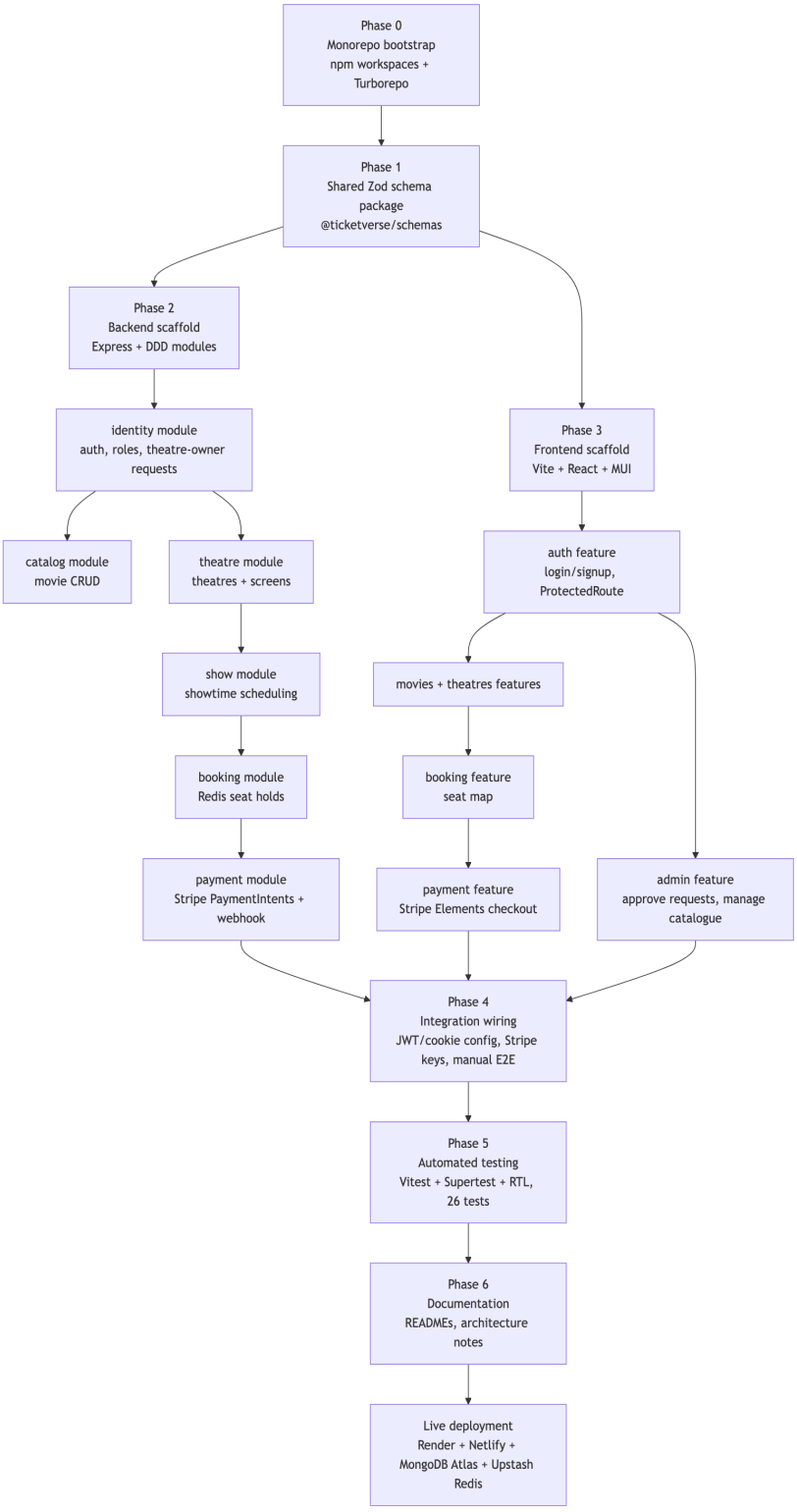

Figure 2.01

Figure 2.01 — Development Process Flow

System Architecture

The deployed system consists of a static single-page React application served from Netlify, a stateless Express API deployed on Render, a managed MongoDB Atlas cluster for durable storage, and a managed Upstash Redis instance used exclusively as an atomic, expiring lock store for in-progress seat holds — never as a system of record. Stripe sits outside the platform's own infrastructure entirely: the API only ever creates a PaymentIntent and verifies an inbound, cryptographically signed webhook event; raw card data never touches TicketVerse's own servers.

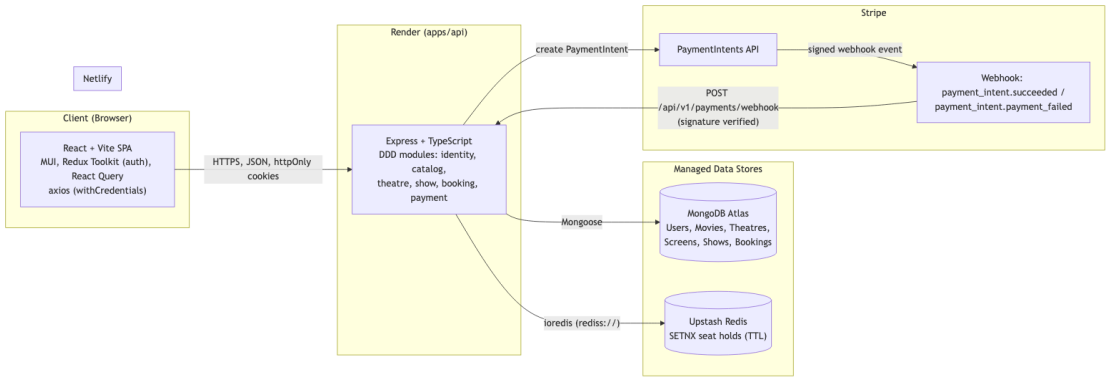

Figure 2.02

Figure 2.02 — System Architecture

Module Breakdown (Domain-Driven Design)

The backend (apps/api/src/modules/) is organized as six bounded contexts, each following the same four-layer internal structure — domain/ (framework-free business rules and entities), application/ (use-case functions that orchestrate domain logic), infrastructure/ (Mongoose models and repository implementations, plus any adapters to other modules), and interface/http/ (Express routes and controllers that translate HTTP requests into use-case calls). This consistent shape means any contributor who understands one module already knows where to look in every other module.

Table 2.01 — Bounded Context / Module Summary

Module	Key Domain Entities	Representative Use Cases	Notable Infrastructure
identity	User, TheatreOwnerRequest	signup, login, refreshTokens, logout, requestTheatreOwner, reviewTheatreOwner Request	MongoUserRepository, MongoTheatreOwner RequestRepository
catalog	Movie	create/list/update/delete movie (admin-only writes)	MongoMovieRepository
theatre	Theatre, Screen	create theatre, define screen seat layout	MongoTheatreRepository, MongoScreenRepository, plus lookup/provisioning adapters used by other modules
show	Show	schedule a show (movie × theatre × screen × time)	MongoShowRepository, ShowLookupAdapter
booking	Booking, seatLayout (value logic)	hold seats, confirm booking, release expired holds	MongoBookingRepository, RedisSeatHoldAdapter (the atomic seat-lock mechanism), BookingConfirmation Adapter
payment	Payment	create PaymentIntent, handle Stripe webhook	MongoPaymentRepository

This module boundary is not merely organizational — it is enforced at the dependency level: the booking module depends on show and theatre only through narrow, purpose-built lookup adapters (e.g., ShowLookupAdapter, ScreenLookupAdapter) rather than importing those modules' Mongoose models directly. This means, for example, that the booking domain logic never needs to know *how* a show is stored — only that it can ask "does this show exist, and what screen does it use?" through a small, stable interface. This is the same dependency-inversion discipline that Domain-Driven Design and hexagonal/ports-and-adapters architecture both recommend, and it is what keeps the RedisSeatHoldAdapter swappable in tests (replaced with ioredis-mock, discussed further in the Testing Strategy chapter) without touching a single line of business logic.

Requirement Gathering

Requirement Gathering Methodology

Because this is an individually built academic project rather than a client engagement, requirements could not be gathered through stakeholder interviews in the traditional sense. Instead, a structured, three-step elicitation process was used to arrive at the functional and non-functional requirements below, chosen specifically to keep the requirements grounded in how real commercial ticketing platforms behave rather than in an idealized or convenient design:

1. **Domain research against real platforms.** The booking flows of well-known ticketing and travel platforms (cinema chains, airline seat selection, train reservation systems) were studied to identify the invariants they all share: a seat/berth/slot is a uniquely identifiable, non-substitutable resource; availability must be shown close to real-time; and a booking is only final once payment clears, not the moment a seat is clicked. These invariants were converted directly into NFR-01 and NFR-04.
2. **Role and persona derivation.** Starting from the single "moviegoer" persona, the two supply-side and platform-operator personas (theatre owner, administrator) were derived by asking, for each piece of data the moviegoer depends on (movie listings, showtimes, seat layouts), *who is responsible for keeping that data correct, and what is the minimum access that role needs to do so without being able to affect data outside their own scope.* This directly produced the role-scoping requirements FR-08–FR-13 and the authorization design described in the Security chapter.
3. **User-story-to-endpoint traceability.** Each functional requirement below was written as a short user story ("As a <*role*>, I want to <*action*>, so that <*outcome*>") and then immediately mapped to a concrete backend endpoint before any UI was built, following

an API-first discipline. This traceability is preserved in Table 3.03 below, so every requirement in this chapter can be checked against a real, tested route rather than remaining an unverified aspiration.

This process deliberately favours requirements that are *falsifiable by a test* — for example, NFR-01 (“two concurrent holds on the same seat must never both succeed”) is not a vague quality goal; it is exactly the assertion made by the concurrent-booking automated test described in the Testing Strategy chapter.

Functional Requirements

Functional requirements describe what the system must *do*, organized by the three user roles the platform serves.

Table 3.01 — Functional Requirements

ID	Requirement	Actor
FR-01	A visitor can create an account (email + password) and log in.	User
FR-02	A logged-in user can browse and search movies, and view showtimes filtered by movie, city, and date.	User
FR-03	A user can view a real-time seat map for a chosen showtime, reflecting both confirmed bookings and seats currently held by other users.	User
FR-04	A user can select one or more available seats and place a temporary hold on them before paying.	User
FR-05	A user can pay for a held booking through Stripe; on successful payment the booking is confirmed automatically without further user action.	User
FR-06	A user can view their own booking history.	User
FR-07	A user can submit a request to become a theatre owner.	User

ID	Requirement	Actor
FR-08	A theatre owner can view and update the theatres they own.	Theatre Owner
FR-09	A theatre owner can create, list, and delete screens (with a seat layout of rows/columns and seat type) for their own theatres only.	Theatre Owner
FR-10	A theatre owner can schedule, update, and cancel showtimes for their own screens, selecting a movie from the shared catalogue.	Theatre Owner
FR-11	An administrator can approve or reject theatre-owner requests; approval promotes the requester's role and provisions their theatre record.	Administrator
FR-12	An administrator has exclusive rights to create, update, and delete entries in the movie catalogue.	Administrator
FR-13	An administrator can view a paginated oversight list of all users, all theatres, and all bookings on the platform.	Administrator

Non-Functional Requirements

Table 3.02 — Non-Functional Requirements

ID	Category	Requirement
NFR-01	Concurrency correctness	Two users attempting to hold the same seat concurrently must never both succeed; the losing request must receive a clear, actionable conflict response.
NFR-02	Security	Passwords must never be stored in plaintext; authentication tokens must never be readable by client-side JavaScript.
NFR-03	Security	All state-changing endpoints must validate their input against a shared schema before touching the database, rejecting malformed or malicious payloads.
NFR-04	Availability of payment truth	Booking confirmation must depend on a signed event from the payment provider, not on the client reporting "payment succeeded" itself.
NFR-05	Portability	The system must run identically against local, containerless development infrastructure (native MongoDB/Redis) and managed cloud infrastructure

ID	Category	Requirement
		(MongoDB Atlas/Upstash), with no code changes — only environment variables should differ.
NFR-06	Testability	Core business rules (auth, RBAC, booking concurrency) must be covered by automated tests that run without any real external network dependency.
NFR-07	Maintainability	Business logic must be isolated from Express/Mongoose so that domain rules can be unit-tested and reasoned about independently of the HTTP framework.
NFR-08	Usability	Every data-driven page must present a distinct loading state, error state (with retry), and empty state, rather than a blank screen on failure or absence of data.

Users and Use Cases

TicketVerse has three actors, each mapped to a distinct set of use cases enforced server-side by role-based middleware rather than merely hidden in the UI:

- **User** (default role on signup) — the moviegoer: browses, books, and pays.
- **Theatre Owner** (elevated via an admin-approved request) — manages their own theatres, screens, and showtimes only; cannot touch the shared movie catalogue.
- **Administrator** — curates the shared movie catalogue, reviews theatre-owner applications, and holds read-only oversight across the whole platform.

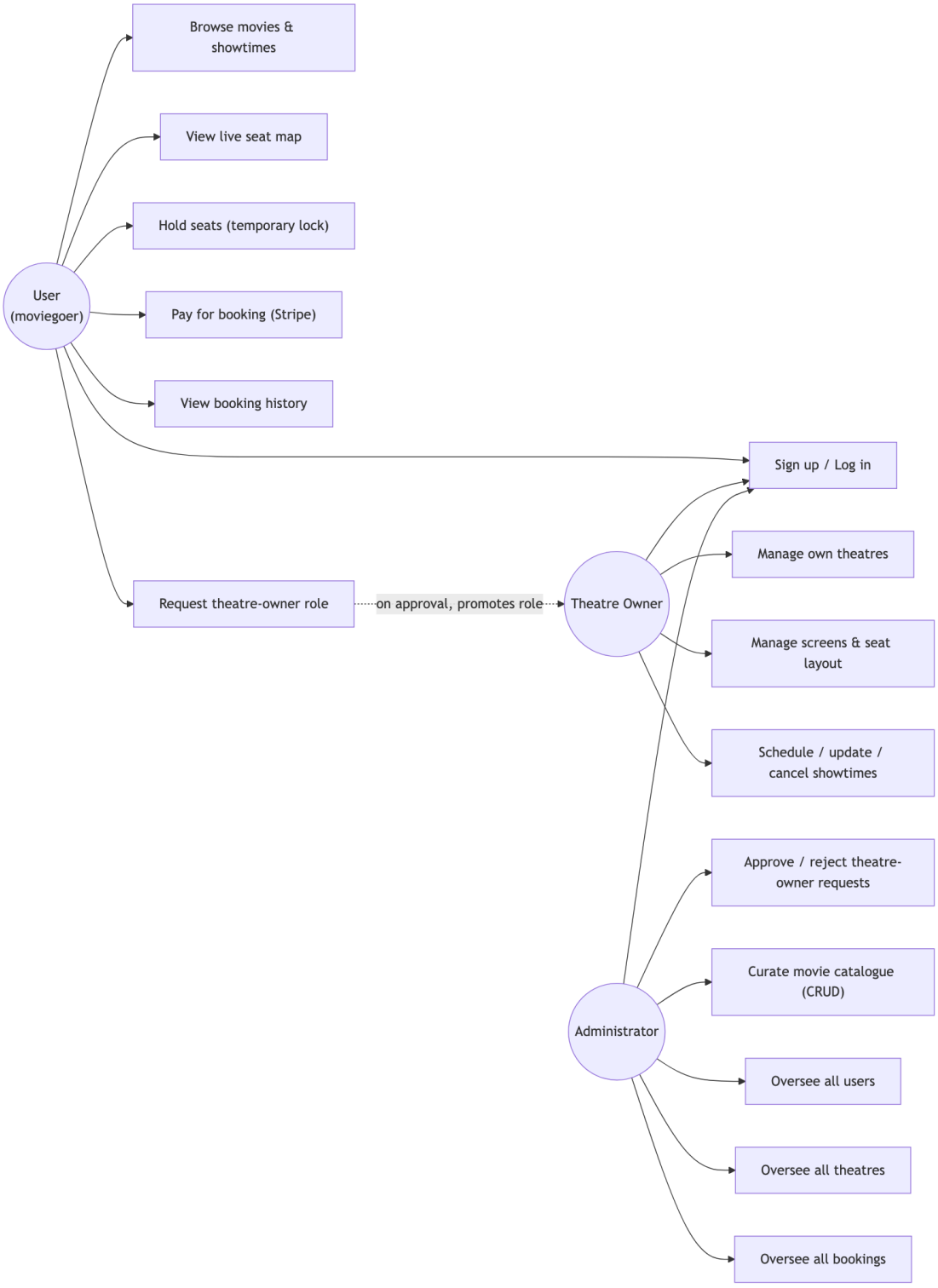

Figure 3.01

Figure 3.01 — Use Case Diagram

Detailed Use Case Descriptions

The following use cases were selected for detailed description because each one exercises a distinct, non-trivial requirement from Table 3.01 and Table 3.02 — concurrency safety, asynchronous payment confirmation, and scoped authorization, respectively — rather than being a routine CRUD operation.

Table 3.04 — Use Case: Hold and Book Seats (UC-04/UC-05)

Field	Description
Actor	User
Preconditions	User is authenticated; the target show exists and has not started; at least one requested seat is not already held or booked.
Main Flow	1. User requests a hold on a set of seat IDs for a show. 2. The API attempts an atomic Redis SETNX-with-TTL lock for every requested seat in a single pass. 3. If all locks succeed, a pending Booking record is created and the held seat IDs and hold-expiry timestamp are returned to the client. 4. The client requests a Stripe PaymentIntent for the held booking. 5. The client confirms payment via Stripe Elements. 6. Stripe sends a signed webhook event to the API. 7. The API verifies the signature, marks the Payment and Booking as confirmed, and releases nothing (the seats remain permanently assigned to this booking).
Alternate Flow (conflict)	At step 2, if any single seat's lock acquisition fails, the entire hold attempt is rejected as a unit (no partial holds) and the user receives a 409-style conflict response naming the unavailable seats.
Alternate Flow (abandonment)	If the user never completes payment, the Redis

Field	Description
	key's TTL expires automatically, releasing the seats back to the pool without requiring a background cleanup job.
Postconditions	Either the booking is confirmed and the seats are permanently unavailable to all other users, or the hold expires and the seats return to the available pool.

Table 3.05 — Use Case: Request and Approve Theatre Ownership (UC-08/UC-13)

Field	Description
Actor(s)	User (requester), Administrator (approver)
Preconditions	Requester has an active User-role account; no pending request already exists for that user.
Main Flow	1. User submits a theatre-owner request (with theatre details). 2. Request is stored in a pending state, visible only to administrators. 3. Administrator reviews the pending request queue. 4. Administrator approves the request. 5. The user's role is promoted to theatre_owner, a Theatre record is provisioned from the request's details, and the user's tokenVersion is incremented server-side.
Alternate Flow (rejection)	At step 4, the administrator instead rejects the request; the requester's role is left unchanged and no theatre is provisioned.
Alternate Flow (forced re-authentication)	Because JWTs are stateless, the promoted user's *existing* access token still carries the old role until it expires (at most 15 minutes) or until they next refresh, at which point the refresh handler detects the tokenVersion bump and issues new tokens carrying the updated role

Field	Description
	— this is the mechanism, not a bug, by which a mid-session role promotion is safely propagated without a full logout.
Postconditions	Approved requester can now create/manage their own theatres, screens, and shows; rejected requester remains a User.

Table 3.06 — Use Case: Scoped Theatre Management (UC-09/UC-10)

Field	Description
Actor	Theatre Owner
Preconditions	Actor holds the theatre_owner role and owns at least one Theatre record.
Main Flow	1. Owner requests their own theatre list. 2. The API filters the query by the authenticated user's ID as the theatre's owner field — never by a client-supplied theatre ID alone. 3. Owner creates a screen (seat layout) or a show against one of their own theatres.
Alternate Flow (scope violation)	If the owner attempts to modify a theatre, screen, or show whose owner field does not match their own user ID, the request is rejected with a 403 regardless of whether the target resource exists, preventing both unauthorized writes and existence-leakage of other owners' resources.
Postconditions	Owner's own inventory is updated; no other owner's data is visible or modifiable.

Feature Set

The table below maps each implemented feature to its concrete HTTP surface in apps/api, confirming that every use case above has a corresponding, tested backend endpoint rather than existing only as a UI mock.

Table 3.03 — Feature Set to API Mapping

Feature Area	Endpoint(s)	Access
Authentication	POST /auth/signup, POST /auth/login, POST /auth/refresh, POST /auth/logout, GET /me	Public / cookie-authenticated
Theatre-owner onboarding	POST /theatre-owner-requests, POST /theatre-owner-requests/:id/review, GET /admin/theatre-owner-requests	User / Admin
Movie catalogue	GET /movies, GET /movies/:id, POST /PATCH /DELETE /movies(/:id)	Public read / Admin write
Theatres & screens	GET /theatres/mine, GET /theatres/:id, PATCH /theatres/:id, GET/POST /theatres/:theatreId/screens, DELETE /screens/:id	Owner-scoped
Showtimes	GET /shows, GET /theatres/:theatreId/shows, GET /shows/:id, POST/PATCH/DELETE /shows(/:id)	Public read / Owner write
Seat availability & holds	GET /shows/:showId/seats, POST /bookings/hold	Authenticated
Bookings	POST /bookings, GET	Authenticated / Admin

Feature Area	Endpoint(s)	Access
	/bookings/mine, GET /bookings/:id, GET /admin/bookings	
Payments	POST /payments/intent, POST /payments/webhook	Authenticated / Stripe-signed
Admin oversight	GET /admin/users, GET /admin/theatres, GET /admin/bookings	Admin

Payments Integration

How Payment Gateways Work

A payment gateway sits between a merchant's application and the card networks/banks, so that the merchant never has to handle raw card numbers directly. In practice this means the merchant's frontend collects card details inside an *iframe or SDK component hosted by the gateway itself* (so the card number never enters the merchant's own JavaScript or servers), the merchant's backend asks the gateway to create a payment "intent" for a given amount, and the gateway confirms — asynchronously, via a signed server-to-server webhook — whether the charge actually succeeded. TicketVerse uses **Stripe** as its payment gateway, specifically the **PaymentIntents API** together with **Stripe Elements** on the frontend (@stripe/react-stripe-js), which is Stripe's documented, PCI-DSS-scope-reducing integration pattern for custom checkout UIs (Stripe, n.d.-a).

The Payment Flow Implemented in TicketVerse

The end-to-end flow, from seat selection to a confirmed booking, is:

1. The user selects seats for a showtime; the frontend calls POST /bookings/hold, which atomically reserves each seat in Redis using SETNX with a time-to-live, so a seat can never be held by two users at once (see Ch.5 and the Requirement Gathering chapter's NFR-01).
2. On a successful hold, the frontend calls POST /bookings, which creates a Booking document in MongoDB with status: "pending_payment".
3. The frontend calls POST /payments/intent, and the backend's makeCreatePaymentIntent use-case (apps/api/src/modules/payment/application/paymentUseCases.ts) calls stripe.paymentIntents.create({...}), persists a Payment record keyed by stripePaymentIntentId, and returns Stripe's client_secret to the frontend.
4. The frontend mounts Stripe's PaymentElement using that client_secret and calls stripe.confirmPayment(...) — at no point does card data pass through TicketVerse's own servers.
5. Stripe sends a webhook event (payment_intent.succeeded or payment_intent.payment_failed) to POST /payments/webhook. The backend's makeHandleStripeWebhook use-case verifies the event's signature with stripe.webhooks.constructEvent(rawBody, signature, STRIPE_WEBHOOK_SECRET)

— rejecting the request with a ValidationError if the signature does not match — before ever trusting the event body. Only after this verification does the booking flip to confirmed (and the corresponding Redis holds are released) or, on failure, the booking is cancelled and the holds released back to the pool.

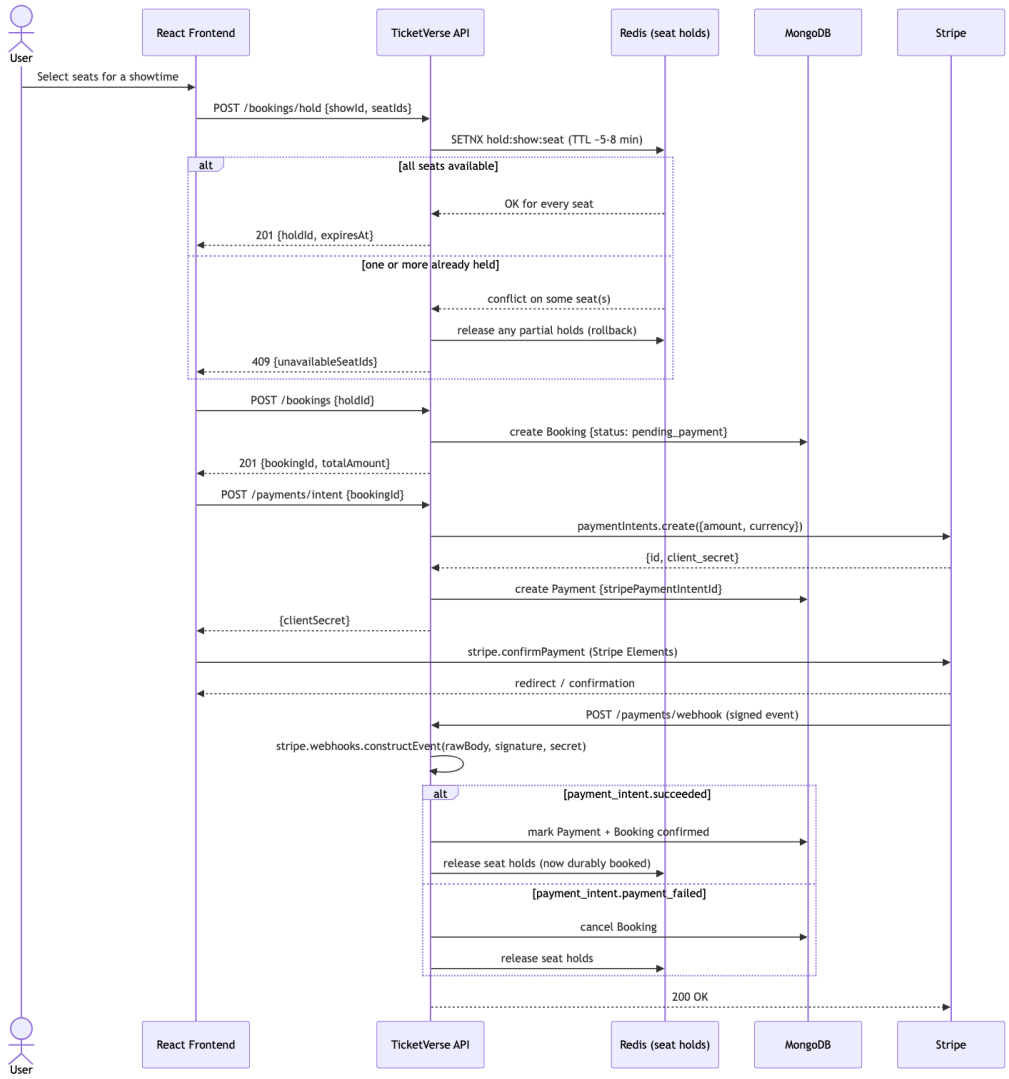

Figure 4.01

Figure 4.01 — Payment Sequence Diagram

Role of Webhooks and HTTP Tunneling

Payment confirmation cannot safely be driven by the client telling the server "I paid" — a malicious or buggy client could claim success for a payment that never happened. Stripe's webhook mechanism solves this by having *Stripe itself* call back into the merchant's server once it has independently confirmed the charge with the card network, and by cryptographically signing that callback so the merchant can verify it genuinely came from Stripe and was not forged or replayed. This is why apps/api/src/app.ts mounts the webhook route *before* express.json(), using Express's raw-body middleware scoped to that single route — Stripe's signature is computed over the exact raw request bytes, and re-serializing a parsed JSON object would invalidate it.

During local development, Stripe cannot reach localhost directly, so the Stripe CLI's stripe listen --forward-to localhost:4000/api/v1/payments/webhook command was used to tunnel Stripe's webhook events to the developer's machine — functionally the same role that a tool like ngrok plays for other webhook-driven integrations, but purpose-built by Stripe for this exact use case (Stripe, n.d.-b). In production, the webhook is registered directly against the live Render URL, so no tunneling is needed once deployed.

PCI DSS and Security Standards

The Payment Card Industry Data Security Standard (PCI DSS) defines security requirements for any system that stores, processes, or transmits cardholder data. By using Stripe Elements to collect card details directly into Stripe-hosted iframes and never routing raw card numbers through TicketVerse's own frontend JavaScript or Express backend, TicketVerse's PCI DSS compliance scope is reduced to **SAQ A**, the lightest self-assessment tier available to merchants who fully outsource card handling to a certified provider (PCI Security Standards Council, n.d.). TicketVerse's own responsibility is limited to keeping its Stripe secret key and webhook signing secret out of source control (both are supplied only via environment variables — see Ch.5 and Ch.6), and to verifying every webhook's signature before acting on it, which the implementation already does.

Table 4.01 — What Touches TicketVerse's Own Servers vs. Stripe

Data / Responsibility	TicketVerse Server	Stripe
Raw card number, CVC, expiry	Never	Collected directly via Stripe Elements
PaymentIntent creation (amount, currency)	Yes (stripe.paymentIntents.create)	Confirms and processes the charge
Card-network authorization decision	No	Yes
Payment success/failure notification	Received via signed webhook only	Originates and signs the event
Storing full card data at rest	Never	N/A (Stripe never stores raw PAN on merchant's behalf either)
Refunds/disputes	Not implemented in v1 (would call Stripe's Refunds API)	Handles fund movement

Alternative Payment Gateways Considered

Stripe was chosen over two commonly used alternatives after weighing each against the project's specific requirements:

- **Razorpay** — the gateway used in the official project template's own worked example, and a natural fit for an India-first audience given its native UPI and net-banking support (Razorpay, n.d.). It was not chosen for this implementation because Stripe's PaymentIntents/Elements documentation and TypeScript SDK typings are more thoroughly documented for a custom React integration, and Stripe's test-mode tooling (the Stripe CLI's listen/trigger commands, discussed below) made local webhook development materially faster to iterate on. The underlying integration pattern — create an intent server-side, collect card data in a gateway-hosted component, confirm via signed webhook — is essentially identical between the two providers, so the architectural lessons transfer directly regardless of which gateway is used in production for a given market.

- **PayPal Checkout** (PayPal, Inc., n.d.) — rejected primarily because PayPal's redirect-based (rather than embedded-element-based) checkout flow does not exercise the same "raw card data must never touch the merchant's own frontend/backend" PCI-scope-reduction pattern as directly, since a meaningful fraction of PayPal transactions do not involve card entry at all, making it a weaker vehicle for teaching PCI DSS scope reduction specifically.
- **A custom/naive gateway simulation** (i.e., mocking "payment success" entirely in-house) — rejected outright, because it would eliminate the single hardest and most instructive problem in the whole project: reconciling an *asynchronous, externally sourced* confirmation event with an already-created, pending domain record, which is precisely the failure mode that makes real payment integrations difficult.

Idempotency and Retry Handling

Two related reliability properties matter for a webhook-driven payment flow, and both are addressed by the implementation:

1. **Webhook delivery is at-least-once, not exactly-once.** Stripe's own documentation states that a webhook endpoint may receive the same event more than once (for example, if the endpoint is slow to respond and Stripe's delivery attempt times out before retrying) (Stripe, n.d.-b). TicketVerse's webhook handler is written to be safe under duplicate delivery: looking up the Payment record by its unique stripePaymentIntentId and only transitioning pending → confirmed once means a second identical webhook delivery for an already-confirmed payment is a harmless no-op rather than a double-processed booking or a duplicate confirmation side effect.
2. **The client-side confirmation call (stripe.confirmPayment) is itself retryable.** If a user's network connection drops after Stripe has received their card details but before the browser learns the outcome, the *authoritative* outcome is still the webhook — the frontend's own confirmation call succeeding or failing is treated as a fast-path UX signal, not as the source of truth for whether the booking is actually confirmed. This is precisely why the webhook, not the confirmPayment promise resolving, is what flips the booking's status in the database (see step 5 of the Payment Flow above).

Testing the Payment Flow

Because a full backend integration test cannot ethically or practically make a real charge against Stripe's live card networks, the payment flow was verified using Stripe's own test-mode tooling rather than being skipped:

- **Stripe's published test card numbers** (e.g., 4242 4242 4242 4242 for a guaranteed-success Visa card, and dedicated numbers for guaranteed declines and 3D Secure challenges) were used to manually exercise both the success and failure branches of the flow end-to-end in the deployed test-mode environment, confirming that a declined card correctly leaves the booking in a cancelled state with the seats released back to the pool.
- **stripe trigger payment_intent.succeeded** (via the Stripe CLI) was used during development to fire synthetic webhook events at the locally tunnelled endpoint without needing to complete a full Elements checkout each time, speeding up iteration on the webhook handler itself.
- The automated backend test suite (see the Testing Strategy chapter) does not call the real Stripe API at all; instead, it exercises the booking module's concurrency guarantees directly, since the payment confirmation step is, by design, a thin, already-battle-tested boundary (Stripe's own signature-verification library) wrapping a simple state transition.

Security

Password Hashing with bcrypt

Storing passwords requires that even if the database is ever compromised, an attacker cannot recover the original passwords. TicketVerse never stores plaintext passwords; instead, apps/api/src/shared/lib/password.ts hashes every password with **bcrypt** at signup (hashPassword) and verifies it at login with a constant-time comparison (comparePassword), using a **salt cost factor of 12**.

Two properties of bcrypt make it suitable for this: first, it automatically generates and embeds a random **salt** into every hash, so two users with the identical password produce entirely different stored hashes — this defeats precomputed **rainbow-table** attacks, which rely on a single hash mapping predictably to a single plaintext across every account. Second, bcrypt is a deliberately **slow, tunable** hashing algorithm (its "cost factor" controls how many rounds of internal key-derivation it performs); at cost factor 12, hashing takes on the order of hundreds of milliseconds, which is negligible for a legitimate login but makes brute-force or dictionary attacks against stolen hashes computationally expensive at scale, unlike a fast general-purpose hash such as SHA-256 that would let an attacker try billions of guesses per second (OWASP, n.d.-a).

JWT Access/Refresh Tokens and Revocation

Authentication state is carried in two JSON Web Tokens (JWTs) rather than a server-side session store: a short-lived **access token** (15 minutes, JWT_ACCESS_TTL) embedding the user's id and role, and a longer-lived **refresh token** (7 days, JWT_REFRESH_TTL) used only to obtain a new access token via POST /auth/refresh. Both are delivered exclusively as **httpOnly, Secure, SameSite** cookies (never returned in a JSON response body and never read by client-side JavaScript), which is the primary defense against token theft via Cross-Site Scripting (XSS) — even if an attacker were able to inject a script into the page, document.cookie cannot read an httpOnly cookie (OWASP, n.d.-b).

Because JWTs are stateless and cannot be individually invalidated once issued, TicketVerse adds a tokenVersion integer field on the User document. Every issued refresh token embeds the tokenVersion value that was current at issuance time; POST /auth/refresh rejects a refresh token whose embedded version does not match the user's *current* tokenVersion. Logging out, and — more importantly — an administrator promoting a user's role (e.g., approving a theatre-owner request), both increment tokenVersion, which immediately revokes every outstanding refresh

token for that user and forces re-authentication, ensuring a promoted user's *next* access token reflects their new role rather than a stale, cached one.

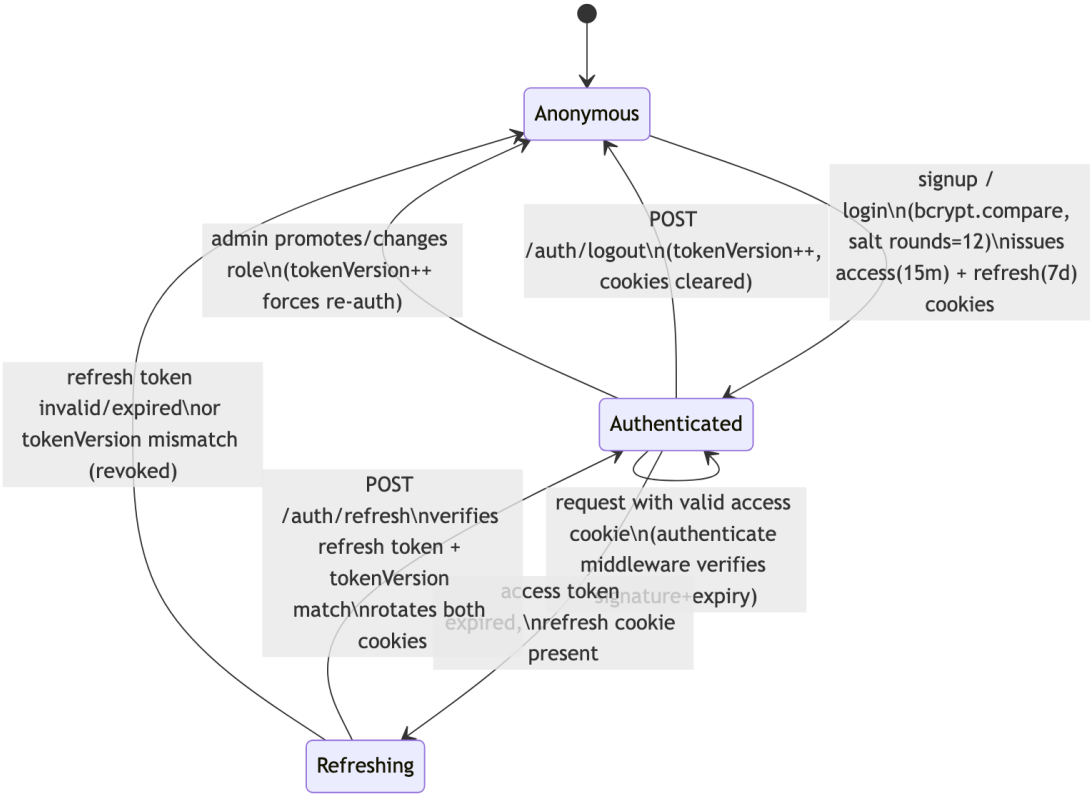

Figure 5.01

Figure 5.01 — Token Lifecycle

Defense-in-Depth Middleware

Beyond authentication itself, apps/api/src/app.ts wires several Express security middlewares that each address a distinct, well-known class of web vulnerability:

Table 5.01 — Security Middleware and Its Purpose

Middleware	Purpose
helmet	Sets a battery of HTTP response headers (e.g., X-Content-Type-Options, disabling X-Powered-By) that reduce the platform's fingerprintability and block several classes of browser-side attacks by default.
cors (with an explicit origin allow-list and credentials: true)	Ensures only the known frontend origin can make cookie-carrying cross-site requests to the API, rather than allowing any website to call the API on a logged-in user's behalf.
express-rate-limit (stricter limiter on /api/v1/auth/*)	Blunts brute-force login/signup attempts by capping the request rate per client IP; app.set("trust proxy", 1) was additionally required in production so the limiter reads the real client IP from Render's X-Forwarded-For header rather than treating every request as coming from Render's own proxy IP.
express-mongo-sanitize	Strips any request key beginning with $ or containing ., preventing NoSQL injection attacks where a malicious JSON body tries to smuggle a MongoDB query operator (e.g., {"$gt": ""}) into a field that is expected to be a plain string.
Zod validation (validate middleware, shared @ticketverse/schemas)	Rejects any request body/query/params that do not conform to the exact expected shape and types *before* any handler or database call runs, closing off a large class of malformed-input

Middleware	Purpose
	and type-confusion bugs.
requireRole(...)	Re-checks the authenticated user's role (from the verified JWT, and for sensitive role-changing endpoints, also against the live database record) before allowing access to owner- or admin-scoped routes — the actual authorization boundary, not just a UI-level hide/show.

OWASP Top 10 Mapping

The OWASP Top 10 is the industry-standard reference list of the most critical web application security risks (OWASP, n.d.-d). Rather than treat security as a single generic "add some middleware" checkbox, each applicable risk category was considered individually against TicketVerse's actual implementation:

Table 5.02 — OWASP Top 10 (2021) Risk Mapping

OWASP Category	Applies?	TicketVerse Mitigation
A01: Broken Access Control	Yes	requireRole middleware plus per-resource ownership checks (e.g., a theatre owner can only mutate theatres/screens/shows whose owner field matches their own user id) enforced server-side, never trusted from the client.
A02: Cryptographic Failures	Yes	Passwords hashed with bcrypt (never reversible encryption); JWTs signed with distinct, sufficiently long (≥16-character-minimum, Zod-enforced) access and refresh secrets; all cookies flagged

OWASP Category	Applies?	TicketVerse Mitigation
		Secure in production so they are never sent over plain HTTP.
A03: Injection	Yes	express-mongo-sanitize blocks NoSQL operator injection; Zod schema validation rejects any field of the wrong type or shape before it reaches a database query; Mongoose's parameterized query builders are used throughout instead of any raw/string-concatenated query construction.
A04: Insecure Design	Yes	The seat-hold-then-confirm design and asynchronous, signature-verified payment confirmation (rather than trusting a client-reported "payment succeeded" flag) are themselves security-relevant design decisions, addressed architecturally rather than patched on afterward.
A05: Security Misconfiguration	Yes	helmet sets secure-by-default headers; environment-variable validation fails fast on missing secrets rather than silently running with defaults; the trust proxy fix (Ch.6) is itself an example of a misconfiguration risk that was identified and corrected rather than left in a

OWASP Category	Applies?	TicketVerse Mitigation
		permissive, insecure default.
A06: Vulnerable and Outdated Components	Yes	Dependencies are pinned via package-lock.json and periodically checked with npm audit (see Dependency Security below); the monorepo's small, well-maintained dependency surface (Express, Mongoose, Stripe SDK, ioredis, jsonwebtoken, bcrypt) was a deliberate choice over less-maintained alternatives.
A07: Identification and Authentication Failures	Yes	Custom-built bcrypt+JWT auth with httpOnly/Secure/SameSite cookies, short-lived access tokens, and version-based refresh-token revocation (see above) directly targets this category.
A08: Software and Data Integrity Failures	Yes	Stripe webhook payloads are cryptographically signature-verified (stripe.webhooks.constructEvent) before being trusted, specifically to prevent a forged or replayed "payment succeeded" event from being accepted as genuine.
A09: Security Logging and	Partial	Programming errors are logged via console.error in the

OWASP Category	Applies?	TicketVerse Mitigation
Monitoring Failures		centralized error handler (see Ch.7/Testing chapters), captured in Render's platform log stream; however, TicketVerse does not yet have a dedicated structured-logging library (e.g., pino/winston) or a security-event audit trail — flagged explicitly as a Limitation in the Conclusion chapter.
A10: Server-Side Request Forgery (SSRF)	Not applicable	TicketVerse's backend does not make any outbound HTTP request based on user-supplied URLs or hostnames (its only outbound calls are to the fixed, hardcoded Stripe API and its own managed MongoDB/Redis instances), so this category does not have a meaningful attack surface in the current feature set.

Input Validation Deep Dive

Every state-changing endpoint validates its request body, query parameters, and route parameters against a Zod schema defined once in the shared packages/schemas package and imported by *both* the Express validate middleware on the backend and the React Hook Form resolvers on the frontend. This single-source-of-truth approach closes a specific, common real-world bug class: a frontend and backend whose validation rules silently drift apart over time (for example, the frontend enforcing a minimum password length that the backend no longer checks after a refactor). Because the same schema object is imported on both sides, such drift is structurally impossible — a change to a schema's rules takes effect on both the client-side form validation

and the server-side request guard simultaneously, and a missed update on either side would be caught immediately by TypeScript's compiler rather than silently shipping.

Validation failures are surfaced as a structured ValidationError (see the error-handling discussion in the Testing Strategy and Technologies Used chapters) carrying field-level details, which the frontend renders next to the relevant form field rather than as an opaque "something went wrong" message — this is a usability property (NFR-08) that falls directly out of the security-motivated validation architecture.

Dependency Security

Because the project depends on third-party npm packages for cryptography (bcrypt, jsonwebtoken), payment processing (stripe), and web-framework plumbing (express, mongoose, ioredis), the dependency tree itself is part of the application's security surface (OWASP Top 10 category A06 above). npm audit (npm, Inc., n.d.) was run periodically against the workspace to check for known vulnerabilities in the resolved dependency graph, and package-lock.json is committed to source control so every install (including on Render's build servers) resolves to the exact same audited set of transitive dependency versions rather than silently picking up a newer, unaudited patch release at deploy time.

Secrets Handling

No secret (JWT signing keys, Stripe secret key, Stripe webhook signing secret, database connection strings) is ever committed to source control; all are read from environment variables validated at startup by a Zod schema (apps/api/src/shared/config/env.ts), which fails fast with a clear error if a required secret is missing, rather than booting into an insecure or broken state. In the deployed environment, these are supplied through Render's dashboard-managed environment variables (marked sync: false in render.yaml, meaning Render prompts for the value once and never re-syncs it from the repository).

Testing Strategy

Philosophy: Testing the Properties That Matter Most

Rather than pursuing blanket code coverage as an end in itself, TicketVerse's test suite is deliberately concentrated on the two classes of behaviour that are hardest to get right by inspection alone and most damaging to get wrong in production: **concurrency correctness** in the booking flow, and **authentication/authorization correctness** in the identity flow. A UI form that renders a label in the wrong place is a cosmetic bug; a race condition that lets two customers pay for the same seat, or a role check that can be bypassed, is a trust-destroying one. The test suite's shape reflects that priority ordering.

Backend Testing (apps/api)

The backend test suite uses **Vitest** (a Jest-compatible test runner with native TypeScript/ESM support and a faster watch mode) together with **Supertest** for issuing real HTTP requests against the Express app in-process, without needing a running network server.

Two infrastructure dependencies are replaced with fast, deterministic in-memory equivalents so the suite can run entirely offline, in CI-free environments, and on any developer's machine without provisioning real cloud resources:

- **mongodb-memory-server** downloads and runs a real, temporary mongod binary in-process. apps/api/test/globalSetup.ts starts exactly **one** shared instance for the entire test run (rather than one per test file), which was a deliberate performance and stability decision — spinning up a fresh mongod per file would multiply both startup latency and the chance of port-binding flakiness. Because a shared instance persists data across test files, apps/api/test/setup.ts clears every collection in an afterEach hook, so each individual test still starts from a known-empty state without needing per-test infrastructure teardown.
- **ioredis-mock** is swapped in for the real ioredis client via vi.mock("ioredis"), so the exact same RedisSeatHoldAdapter code exercised in production runs against an in-memory Redis-compatible implementation during tests — the SETNX-with-TTL locking logic under test is the real production code path, not a stubbed-out fake of it.

Table 6.01 — Backend Test Files and Coverage Focus

File	Test Cases	Focus
test/identity.integration.test.ts	5	Full signup → session → refresh → logout flow; password rejection; invalid/stale refresh token rejection; missing-cookie rejection; RBAC enforcement on a protected route
test/booking.integration.test.ts	1	End-to-end seat hold + booking, and — critically — that two concurrent hold requests for the *same* seat resolve to exactly one success and one rejection, never two successes
test/jwt.unit.test.ts	4	Access and refresh token sign/verify round-trips; a tampered token is rejected; a token signed with the wrong secret is rejected
test/password.unit.test.ts	4	bcrypt hashing produces a verifiable hash; two hashes of the identical password differ (unique salts); correct password verifies; incorrect password is rejected

The single concurrency test in booking.integration.test.ts is disproportionately important relative to its count of one: it is the automated, repeatable proof that NFR-01 (no seat is ever double-booked) holds, executed by firing two hold requests for an overlapping seat set at effectively the same time via Promise.all and asserting that exactly one resolves successfully. Without this test, the concurrency-safety claim made throughout this report would rest entirely on manual testing

and code review rather than on a machine-checked guarantee that re-runs on every future change to the booking module.

Test execution is configured with fileParallelism: false in the Vitest config, meaning test files run sequentially rather than in parallel worker processes — a direct consequence of sharing one mongodb-memory-server instance across the whole run; parallel file execution against a single shared database would reintroduce exactly the kind of test-order-dependent flakiness the in-memory-database strategy was chosen to eliminate.

Frontend Testing (apps/web)

The frontend test suite uses **Vitest** with **@testing-library/react** (rendering components into a jsdom environment and asserting on behaviour from the user's perspective — what is rendered and what happens on interaction — rather than on internal component state) and **@testing-library/user-event** for realistic simulated user input.

Table 6.02 — Frontend Test Files and Coverage Focus

File	Test Cases	Focus
ProtectedRoute.test.tsx	5	Loading state while auth status resolves; unauthenticated user is redirected; a user without an allowed role is denied; an authorized user renders the protected content; a route with no allowedRoles restriction renders for any authenticated user
AuthForms.test.tsx	4	Login form surfaces validation errors on invalid input and submits correctly on valid input; signup form enforces password rules and submits correctly on valid input
SeatMapPage.test.tsx	3	Seat selection toggles on click;

File	Test Cases	Focus
		the client-side MAX_SEATS cap is enforced; a seat already marked booked/held cannot be selected

ProtectedRoute.test.tsx is the frontend analogue of the backend's RBAC test: it verifies that the *same* role-gating logic the backend enforces authoritatively is also correctly mirrored in the client's route guard, so that an unauthorized user is never even shown a UI affordance for an action the backend would reject anyway — a defense-in-depth, UX-quality property rather than a security boundary in itself (the security boundary is, and must remain, the backend's requireRole check, discussed in the Security chapter).

What Is Deliberately Not Covered

No coverage-percentage target was set, and several categories of testing were consciously deferred rather than attempted incompletely: full end-to-end browser testing (e.g., Playwright/Cypress driving a real browser against the deployed site), load/performance testing of the Redis-based seat-hold mechanism under high concurrent volume beyond the two-request race condition already covered, and exhaustive UI snapshot testing of every admin panel. These are named explicitly as Limitations in the Conclusion chapter rather than silently omitted, consistent with this report's overall preference for grounded, verifiable claims over implied completeness.

Deployment Flow

Source Control

TicketVerse's entire history — from the initial monorepo bootstrap commit through every backend module, frontend feature, security fix, and deployment configuration change — is kept in a single Git repository on GitHub (aravindkarteekr/ticketverse, main branch), following the Conventional Commits format (type(scope): description) so the commit log itself is a readable, chronological record of the build process referenced throughout this report.

Live Deployment Topology

The application is deployed across four managed, free-tier cloud services rather than described only hypothetically:

- **Backend API** — deployed on **Render** as a web service, live at https://ticketverse-api.onrender.com, built and started via a render.yaml Blueprint at the repo root (Render, n.d.). Render only picks up render.yaml when a service is created through its **Blueprint** flow specifically — creating a plain "Web Service" ignores the file entirely, which was confirmed directly during this project's own deployment.
- **Frontend** — deployed on **Netlify** as a static single-page application, live at https://ticketverse-web.netlify.app, built via netlify.toml at the repo root with a client-side routing rewrite (/* -> /index.html, status 200) so React Router's deep links resolve correctly on refresh (Netlify, n.d.).
- **Database** — **MongoDB Atlas** free-tier (M0) cluster, connected via mongodb+srv:// SRV-record connection string.
- **Cache / lock store** — **Upstash Redis** free-tier instance, connected via a rediss:// (TLS) URL that ioredis auto-detects and upgrades to a TLS connection with no code changes.
- **Payment gateway** — **Stripe**, in test mode, with a live webhook endpoint registered against the deployed Render URL (https://ticketverse-api.onrender.com/api/v1/payments/webhook).

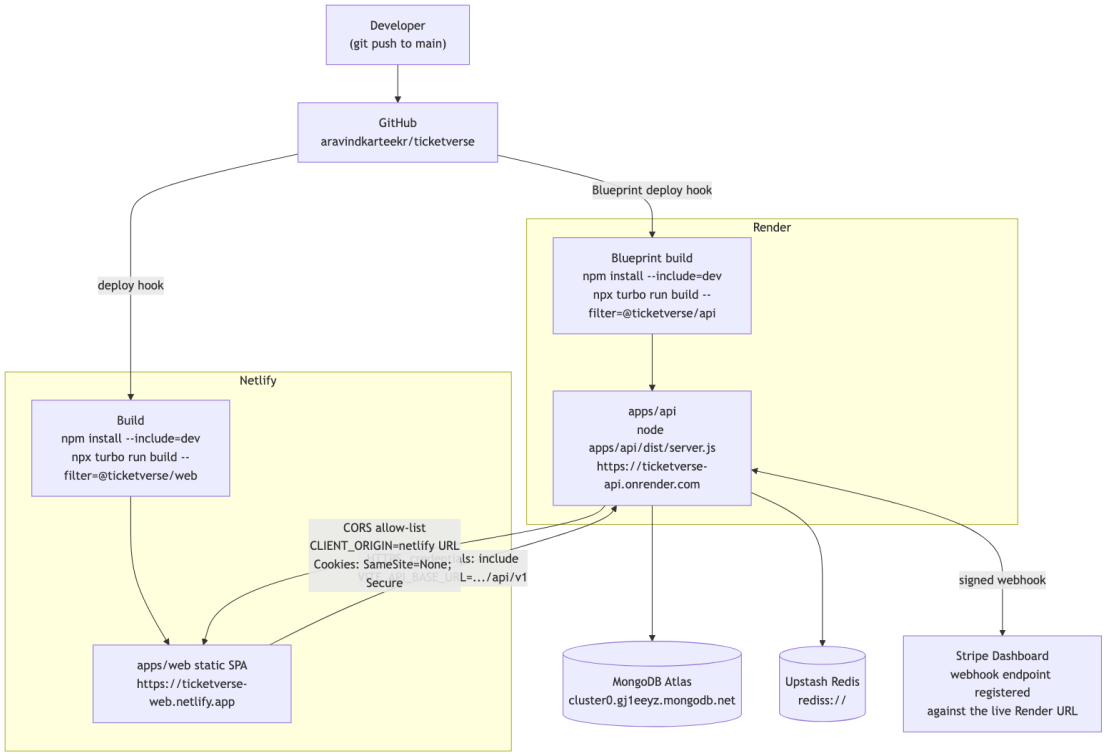

Figure 6.01

Figure 6.01 — Deployment Architecture

Build Configuration Gotcha: NODE_ENV=production and devDependencies

Both Render and Netlify set NODE_ENV=production automatically as a build-time environment variable. By default, npm install under NODE_ENV=production skips devDependencies entirely — which broke the very first deploy attempt, because this monorepo's build tools (tsup, typescript, vite) are all declared as devDependencies, not production dependencies. The fix, applied identically in both render.yaml and netlify.toml, was to change the build command to explicitly request dev dependencies:

npm install --include=dev && npx turbo run build --filter=@ticketverse/api

This is a generalizable lesson for any Node.js monorepo deployed to a platform that sets NODE_ENV=production at build time: the *build* step still needs devDependencies (compilers, bundlers), even though the *runtime* step correctly should not.

CORS and Cross-Site Cookies

Because the frontend (ticketverse-web.netlify.app) and backend (ticketverse-api.onrender.com) are served from different origins, two changes were required beyond a purely same-origin local setup:

1. **CORS** — Express's cors middleware is configured with an explicit single allowed origin (CLIENT_ORIGIN, set to the live Netlify URL) and credentials: true, so the browser permits the frontend's axios client (configured with withCredentials: true) to send and receive TicketVerse's authentication cookies cross-site.
2. **Cookie attributes** — in production, TicketVerse's cookie-setting code flips to Secure: true; SameSite: "None", which modern browsers require for any cookie sent on a cross-site request; on localhost, cookies instead use SameSite: "Lax" without Secure, since local HTTP traffic isn't HTTPS. This is driven entirely by an environment flag (COOKIE_SECURE) rather than hard-coded, so no source change was needed between local and production environments.

Environment Variables and Secret Management

render.yaml marks every secret-bearing environment variable (MONGODB_URI, REDIS_URL, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET) as sync: false, which tells Render to prompt for the value once in its dashboard rather than reading it from the (public) repository — meaning no real secret was ever committed to Git or pasted into a shared document. VITE_API_BASE_URL and VITE_STRIPE_PUBLISHABLE_KEY are configured as Netlify build-time environment variables; because Vite bakes environment variables into the built JavaScript bundle at build time (not read at runtime), any change to these values requires a fresh Netlify build/deploy to take effect, which was itself the root cause of one deployment issue encountered (the initial VITE_API_BASE_URL value omitted the /api/v1 path prefix, causing every API call from the deployed frontend to 404 until it was corrected and the site rebuilt).

Deployment Verification

At the time of writing, both the live API health check (GET /health → 200 OK) and the live frontend (GET / → 200 OK) were confirmed reachable, and a request to the live signup endpoint correctly returned a 400 validation error for an empty payload rather than a 404, confirming the API-base-URL and routing configuration are correctly wired end-to-end in production.

CI/CD Approach

TicketVerse does not use a separate GitHub Actions workflow (there is no .github/workflows/ directory in the repository); build and test verification, and deployment, are instead split across two distinct mechanisms:

1. **Local pre-push verification** — before any commit is pushed, the full monorepo check (npx turbo run typecheck lint test build) is run locally across every workspace, using Turborepo's task graph to build and test packages in dependency order and cache unchanged results between runs. This is the gate that catches type errors, lint violations, and failing tests *before* they ever reach main.
2. **Platform-native continuous deployment** — both Render and Netlify are configured (via render.yaml and netlify.toml respectively) to watch the main branch directly and automatically trigger a fresh build and deploy on every push, without any intermediate GitHub Actions orchestration layer.

This is a deliberate trade-off rather than an oversight: for a single-contributor academic project, a dedicated GitHub Actions workflow would duplicate work that Turborepo's local task graph and the two hosting platforms' own native build pipelines already perform, at the cost of additional YAML to maintain and CI minutes to consume. The trade-off this defers is **pre-merge protection on pull requests** — since there are no feature branches merged via reviewed PRs in a solo project, a GitHub Actions workflow's main practical benefit (blocking a broken PR from merging) does not apply here. For a team project or a production system with multiple contributors, adding a GitHub Actions workflow that runs the same turbo run typecheck lint test build command on every pull request would be the natural next step, and is noted explicitly as a possible improvement in the Conclusion chapter.

Monitoring and Health Checks

TicketVerse's operational visibility currently rests on three mechanisms, all already in place rather than hypothetical:

- **A dedicated /health endpoint** (deliberately kept outside the /api/v1 versioned route prefix, and outside the /api/v1/auth rate limiter) that Render polls automatically to determine whether the deployed instance is healthy enough to keep serving traffic, and to gate zero-downtime redeploys — Render will not route traffic to a new deploy until its health check passes.

- **Platform log streams** — both Render and Netlify expose a live build-and-runtime log stream in their respective dashboards; the centralized Express error handler's console.error(err) calls for unhandled/programming errors land directly in Render's log stream, which was used during this project to diagnose the express-rate-limit/trust proxy misconfiguration (Ch.5) from a live production error rather than from a local reproduction.
- **Manual live-endpoint verification** — the curl-based checks described in Deployment Verification above were performed directly against the production URLs at the point of each deployment change, rather than assumed to be correct from the build succeeding alone; a successful build and a successfully *running* application are not the same guarantee, particularly for configuration-only failures (missing environment variables, CORS misconfiguration) that only surface at runtime.

No dedicated uptime-monitoring or alerting service (e.g., a synthetic uptime check with paging) is configured, since both Render and Netlify's free tiers already provide basic platform-level restart-on-crash behaviour, and a single-operator academic project does not have an on-call rotation to page. This is named explicitly as a Limitation in the Conclusion chapter rather than left implicit.

Rollback Strategy

Because both Render and Netlify redeploy directly from the main branch's latest commit, rolling back a bad deployment reduces to a Git operation rather than a bespoke deployment-tool procedure: both platforms retain a history of previous successful deploys in their dashboards and support re-promoting a prior deploy to production with a single click, independent of the current state of main — this was used as a safety net (though not actually needed) during the live-deployment phase of this project, since a broken deploy on either platform can be reverted to the last known-good build without needing to force-push or revert commits in Git first. For a Git-level rollback (as opposed to a platform-dashboard rollback), git revert of the offending commit followed by a normal push is the preferred mechanism, since it preserves history and triggers the same automatic redeploy pipeline as any other change, rather than git reset --hard plus a force-push, which would rewrite shared history unnecessarily for what is, functionally, just another forward-moving commit.

Technologies Used

The MERN Stack

TicketVerse is built on the **MERN** stack — **M**ongoDB, **E**xpress, **R**eact, **N**ode.js — a widely used combination for building full-stack JavaScript/TypeScript applications where the same language (TypeScript, a typed superset of JavaScript) is used from the database layer's query code through to the browser UI.

MongoDB is a document-oriented NoSQL database that stores data as flexible, JSON-like BSON documents rather than fixed-schema rows in tables (MongoDB, n.d.-a). TicketVerse uses **Mongoose** as an Object Document Mapper (ODM) on top of MongoDB, which lets each backend module define an explicit schema in code (e.g., a Booking document with showId, seatIds, status, totalAmount) while still benefiting from MongoDB's flexibility — for example, the Show document embeds per-seat pricing without needing a separate join table. A real-world analogy: MongoDB is closer to a flexible filing cabinet where each folder (document) can have a slightly different internal layout, whereas a traditional relational database is closer to a fixed spreadsheet where every row must have exactly the same columns.

Express is a minimal, unopinionated web framework for Node.js that TicketVerse uses to define its entire HTTP surface — routing, middleware (authentication, validation, rate limiting), and error handling (Express, n.d.). Its middleware-chain model is what makes the security stack described in Ch.5 possible: each request passes through helmet, then cors, then body-parsing, then express-mongo-sanitize, then route-specific authenticate/requireRole/validate middleware, before finally reaching a controller — each concern isolated into its own composable function.

React is a component-based UI library used to build apps/web as a single-page application (React, n.d.). Rather than server-rendering full HTML pages for every navigation, React re-renders only the parts of the page whose underlying data changed — for instance, updating just the seat map's colors when a hold conflict response arrives, without reloading the whole page. This project uses React Router v6's data-router APIs for client-side navigation and route-level error boundaries (errorElement).

Node.js is the JavaScript runtime that executes apps/api outside the browser (Node.js, n.d.). Its single-threaded, non-blocking I/O event loop is particularly well suited to an application like TicketVerse where most work — querying MongoDB, reading/writing Redis, calling Stripe's

API — is I/O-bound rather than CPU-bound: while one request waits on a database round-trip, Node.js is free to keep serving other requests instead of blocking a whole thread on the wait.

Table 7.01 — MERN Stack Summary

Layer	Technology	Role in TicketVerse
Database	MongoDB + Mongoose	Durable storage for users, movies, theatres, screens, shows, bookings, payments
Backend	Express + Node.js	HTTP API, middleware security chain, DDD business logic
Frontend	React + Vite	Single-page application, seat map, checkout, role-based dashboards

Redis

Redis is an in-memory key-value data store (Redis, n.d.), used in TicketVerse exclusively as a **temporary, atomic lock store** for seat holds — never as a system of record. The SETNX (set-if-not-exists) command gives an atomic "only one caller can succeed" guarantee across concurrent requests, which is exactly the primitive needed to prevent two users from holding the same seat simultaneously; combined with a time-to-live (TTL), an abandoned hold (a user who never completes payment) is automatically released without any cleanup job. A real-world analogy: Redis's role here is like a nightclub's wristband counter — it doesn't remember your entire life story (that's MongoDB's job), it just very quickly and reliably answers "has this specific slot already been claimed, right now?"

Stripe

Stripe is the payment gateway integrated in Ch.4, chosen over alternatives such as Razorpay for its mature, well-documented **PaymentIntents** and **Elements** APIs and first-class Node.js/React SDKs (Stripe, n.d.-a). Its webhook-signing mechanism (Ch.4, Ch.5) is a recurring pattern across payment and platform APIs generally: never trust an inbound event until its cryptographic signature has been verified against a shared secret.

Zod and Shared Schema Validation

Zod is a TypeScript-first schema declaration and validation library (Zod, n.d.). TicketVerse defines every domain shape (a movie, a booking request, a signup payload) exactly once in packages/schemas, and both apps/api (as Express validation middleware) and apps/web (as react-hook-form's zodResolver, and to type React Query results) import that single definition. This eliminates an entire category of bugs common to hand-written full-stack projects: the frontend and backend silently disagreeing about what a valid request looks like, because there is only ever one schema, not two independently maintained ones.

Turborepo and npm Workspaces

Turborepo is a build-system orchestrator for JavaScript/TypeScript monorepos, used here on top of **npm workspaces** (Turborepo, n.d.). npm workspaces let apps/api and apps/web depend on packages/schemas as a normal, locally-resolved npm dependency (rather than a published registry package), and Turborepo adds a task graph and caching layer on top so that, for example, npx turbo run build only rebuilds packages/schemas when its source actually changed, and skips rebuilding it (using a cached result) otherwise. This is analogous to a spreadsheet with formulas: change one cell (a shared schema), and only the cells that actually depend on it are recalculated — everything else is served from cache.

Material UI (MUI), Redux Toolkit, and React Query

Material UI (MUI) supplies TicketVerse's component library and theme (buttons, tables, dialogs, form fields) (MUI, n.d.), so the project could focus engineering effort on business logic rather than hand-rolling every UI primitive from scratch.

Redux Toolkit manages a deliberately small slice of *client-only* UI state — specifically the authenticated user's identity and role, hydrated once at app load via GET /me (Redux Toolkit, n.d.). TicketVerse does **not** use Redux for server data (movies, bookings, seat availability); that responsibility belongs entirely to **React Query** (TanStack Query, n.d.), which handles fetching, caching, background refetching, and — critically for this project — exposes isLoading/isError/error/refetch on every query, which is what powers the loading/error/empty-state handling on every data-driven page described in the Requirement Gathering chapter's NFR-08.

Testing Stack: Vitest, Supertest, React Testing Library

The backend is tested with **Vitest** as the test runner, **Supertest** for HTTP-level integration assertions against the real Express app, **mongodb-memory-server** for an ephemeral, disposable MongoDB instance per test run, and **ioredis-mock** to simulate Redis without a real network dependency (Vitest, n.d.; Supertest, n.d.). This combination allows the booking module's genuinely concurrency-sensitive behavior — a deliberate double-hold attempt on the same seat — to be asserted automatically on every run, rather than relying on manual, easy-to-forget exploratory testing. The frontend is tested with Vitest plus **React Testing Library**, which asserts on rendered UI output and user-facing behavior (e.g., a validation error appearing after an invalid form submission) rather than internal component implementation details.

Table 7.02 — Supporting Technology Summary

Technology	Category	Primary Role in TicketVerse
Redis (ioredis)	In-memory data store	Atomic, expiring seat-hold locks
Stripe	Payment gateway	PaymentIntents, Elements checkout, signed webhooks
Zod	Schema validation	One shared request/response contract for backend + frontend
Turborepo + npm workspaces	Monorepo tooling	Shared package resolution, cached incremental builds
MUI	UI component library	Theming and pre-built accessible components
Redux Toolkit	Client state	Authenticated user identity/role only
React Query	Server state	Fetching, caching, loading/error/empty states
Vitest + Supertest + RTL	Automated testing	26 tests across backend and frontend

Challenges and Lessons Learned

This chapter documents the concrete, non-obvious problems encountered while building and deploying TicketVerse, and how each was diagnosed and resolved. Unlike the Limitations section of the Conclusion (which lists what the system deliberately does *not* yet do), this chapter is about problems that *were* solved during development — each one is presented as problem → root cause → fix, because the diagnostic process is often as instructive as the final fix itself.

Challenge 1: Naive Seat Availability Checking Allows Double-Booking

Problem. An early, intentionally naive first draft of the booking logic checked seat availability with a find() query, and — only if no conflicting booking was found — created a new Booking document in a second, separate step. Under concurrent load (two requests for the same seat arriving close together), both requests could pass the availability check before either had written its booking, because the check-then-write sequence is not atomic.

Root cause. MongoDB does not, by itself, provide a way to atomically say "reserve this seat only if it is not already reserved" across a check-then-write pair of separate queries without an explicit locking mechanism; a unique index alone would catch the conflict only at the final booking-confirmation step, after a customer may have already been shown a misleadingly successful "seat held" response and started entering payment details.

Fix. Seat reservation was moved to a Redis SETNX (set-if-not-exists) operation with a time-to-live, executed for every requested seat in a single hold request. SETNX is atomic at the Redis level by construction — Redis is single-threaded for command execution, so two concurrent SETNX calls for the same key can never both report success. This moved the actual concurrency-safety guarantee out of application-level check-then-write logic and into a primitive that is atomic by the database's own execution model, which is a substantially stronger guarantee than any amount of careful-looking application code could provide on its own. This fix, and the automated test that now proves it, are described in the Requirement Gathering and Testing Strategy chapters.

Challenge 2: Stripe Webhook Signature Verification Failing Silently

Problem. During initial webhook integration, every webhook event was rejected with a signature-mismatch error, even though the webhook signing secret was correctly configured.

Root cause. Express's default express.json() body-parser middleware, mounted globally in app.ts, parses the incoming request body into a JavaScript object and discards the original raw bytes.

Stripe computes its signature over the *exact original raw request body*; re-serializing the parsed JSON object (even if logically identical) produces a byte sequence that does not match the signature Stripe computed, because JSON re-serialization is not guaranteed to preserve exact whitespace, key ordering, or numeric formatting.

Fix. The webhook route was mounted with its own dedicated express.raw({ type: "application/json" }) middleware, scoped *only* to that one route, and positioned before the global express.json() middleware in the route registration order so the raw-body parser wins for that specific path. This is a general lesson for integrating any webhook-based, signature-verified API: the exact raw bytes of the request must be preserved and passed to the signature-verification function, which usually means special-casing that one route's body-parsing rather than relying on a single global JSON parser for the whole application.

Challenge 3: Cross-Site Cookies Silently Rejected in the Deployed Environment

Problem. After the initial live deployment (frontend on Netlify, backend on Render — two different origins), login appeared to succeed (a 200 response was received), but the very next request from the browser was treated as unauthenticated, as if no cookie had been set at all.

Root cause. Modern browsers refuse to store or send a cookie set by a cross-site response unless that cookie explicitly carries SameSite=None *together with* the Secure attribute (Mozilla Developer Network, n.d.); a cookie configured only for same-origin local development (SameSite=Lax, no Secure flag, since local development runs over plain HTTP) is silently dropped by the browser in a genuinely cross-site, HTTPS production deployment, with no visible error surfaced to either the client-side JavaScript or the server.

Fix. Cookie attributes were made environment-dependent via a single COOKIE_SECURE flag (see Ch.7), rather than hard-coded: SameSite=None; Secure=true in production (required for the real cross-origin Netlify/Render topology) and SameSite=Lax; Secure=false in local development (required because local HTTP traffic cannot satisfy the Secure attribute's HTTPS-only requirement). The broader lesson: a cross-site cookie configuration that works in a same-origin local setup can fail *completely silently* — no console error, no failed network request — once genuinely deployed across two different domains, which makes this exact class of bug easy to miss without deliberately testing the live, cross-origin deployment rather than only local development.

Challenge 4: Build-Time Environment Variables Baked at the Wrong Time

Problem. After redeploying the frontend with a corrected VITE_API_BASE_URL value in Netlify's dashboard, API calls from the live site continued failing against the *old* URL for a period, even though the dashboard clearly showed the new, correct value saved.

Root cause. Vite, like most modern frontend build tools, inlines import.meta.env.VITE_* variables into the compiled JavaScript bundle **at build time**, not read dynamically at runtime (Vite contributors, n.d.). Updating an environment variable's value in a hosting dashboard has no effect on an already-built, already-deployed bundle — the new value only takes effect on the *next* build.

Fix. Triggering a fresh Netlify build/deploy (rather than assuming the dashboard change alone was sufficient) picked up the corrected value. The generalizable lesson, applicable to any Vite/webpack/similar build-time-inlined environment variable setup: changing an environment variable's value in a hosting platform's dashboard is necessary but not sufficient — a rebuild must also be triggered, and this distinction between build-time and runtime configuration is worth explicitly verifying (e.g., by checking the deployed bundle's contents) rather than assumed.

Challenge 5: express-rate-limit Rejecting All Traffic in Production Only

Problem. Shortly after the live deployment, the API began returning validation errors from its rate-limiting middleware on ordinary, legitimate traffic — a failure mode that had never appeared during local development or in the automated test suite.

Root cause. Render's edge proxy sits in front of every deployed web service and sets an X-Forwarded-For header identifying the real client IP; Express's default trust proxy setting is false, meaning Express does not trust *any* proxy-supplied header by default (a deliberately conservative default, since blindly trusting a client-controlled header would let an attacker spoof their apparent IP and bypass IP-based rate limiting entirely). express-rate-limit detected this exact contradiction — a forwarded-for header present, but no trusted proxy configured to explain its presence — and refused to guess, raising a validation error rather than silently making an unsafe assumption.

Fix. app.set("trust proxy", 1) was added, explicitly telling Express to trust exactly **one** hop of proxying — precisely matching Render's actual single-reverse-proxy topology. This was deliberately not set to trust proxy: true (which would trust an unbounded chain of proxies and is a documented misconfiguration risk for exactly the IP-spoofing reason above); the fix is only as

permissive as the real deployment topology requires, following the same minimal-privilege principle applied throughout the Security chapter. This issue was diagnosed directly from a live production error in Render's log stream rather than reproduced locally, since local development has no reverse proxy in front of it and therefore never triggers the condition at all — reinforcing the value of monitoring real production logs (see the Deployment Flow chapter) rather than assuming local-only testing is sufficient.

Cross-Cutting Lesson: Bugs That Only Exist at the Seams

A pattern across four of the five challenges above (2, 3, 4, and 5) is that each bug only manifested at a *seam* between two systems that individually worked correctly in isolation: Express's body parser and Stripe's signature verifier (Challenge 2), the browser's cookie policy and the two-origin deployment topology (Challenge 3), Vite's build pipeline and Netlify's dashboard configuration model (Challenge 4), and Express's trust model and Render's reverse proxy (Challenge 5). None of these would have been caught by unit-testing either side of the seam alone; each required either an integration test that exercised the real seam (as Challenge 1's concurrency test does) or, for the deployment-specific seams, direct verification against the actual live, deployed environment rather than a local approximation of it. This is the single clearest practical lesson this project produced: **local development environments are, by construction, missing exactly the properties (a reverse proxy, a second origin, a real build artifact) that cause an entire class of bugs to exist in the first place**, so a deployment phase that only re-runs the local test suite against a new URL is not actually validating the properties most likely to break.

Conclusion

Key Takeaways

Building TicketVerse end-to-end surfaced several lessons that are easy to state in the abstract but only really internalize by implementing them: that “check availability, then book” is not the same operation as “atomically claim availability” under concurrent load, and that the difference between the two is exactly what separates a system that silently double-books a seat under real traffic from one that does not; that asynchronous, signature-verified webhook confirmation is not a Stripe-specific quirk but the general shape of how any trustworthy payment integration must work, because a client can never be allowed to self-report a successful charge; that role-based access control must be enforced at the server boundary (middleware re-checking the verified token’s role, and for sensitive operations, the live database record) rather than only hidden in the UI, since a determined user can always call the API directly; and that a shared, single-source-of-truth schema (Zod, imported by both ends of the stack) eliminates an entire category of frontend/backend contract-drift bugs that are otherwise very easy to introduce silently over time.

Practical Applications

The seat-hold-then-confirm concurrency pattern implemented here generalizes directly to airline seat selection, train/bus reservation systems, concert and event ticketing, restaurant table reservations, and even limited-inventory e-commerce flash sales — any domain where a scarce, discrete resource must be provisionally reserved for one user while a slower external step (payment) completes. Similarly, the RBAC and admin-oversight pattern (a public-facing role, a privileged “operator” role scoped to their own resources, and a platform-wide administrator role) is a very common shape for multi-sided marketplace and SaaS platforms generally, well beyond ticketing specifically.

Limitations

The current implementation has several known limitations, each identified honestly rather than glossed over: there is no self-service forgot-password/reset flow in v1 (a locked-out demo user must be reset directly via the database); role changes take effect only after the affected user’s next authentication refresh rather than instantly server-push; the production frontend bundle is a single ~735KB JavaScript chunk that has not yet been code-split by route, which Vite’s own build output flags as larger than its recommended threshold; Render’s free tier spins its web service down after a period of inactivity, meaning the very first request after idle can take tens of seconds

to respond (a real, user-visible cold-start cost of the “deploy for free” choice made for this project); and the automated test suite, while covering the core concurrency-sensitive booking path and the authentication/RBAC flows, does not yet include end-to-end browser-level tests (e.g., Playwright/Cypress) of the full checkout journey.

Cost Implications and Improvement Suggestions

Every managed service used — Render’s free web-service tier, Netlify’s free static-hosting tier, MongoDB Atlas’s free M0 cluster, and Upstash Redis’s free tier — has a $0 baseline cost, with Stripe’s only unavoidable cost being its standard per-transaction processing fee on real (non-test-mode) charges. This makes the architecture demonstrated here genuinely viable for an early-stage product to validate demand before committing to paid infrastructure. For a production-scale successor, the clearest next investments would be: code-splitting the frontend bundle by route; adding end-to-end browser tests; introducing a paid, always-on compute tier to eliminate cold starts; and adding a self-service password-reset flow (email delivery via a transactional email provider) that was deliberately scoped out of this academic build to keep the authentication surface area focused on the concepts most relevant to the report — bcrypt hashing and JWT rotation — rather than on SMTP integration.

References

1. Stripe. (n.d.-a). *Accept a payment* [Payment Intents API and Stripe Elements documentation]. Stripe Docs. https://stripe.com/docs/payments/accept-a-payment
2. Stripe. (n.d.-b). *Test webhooks locally with the Stripe CLI*. Stripe Docs. https://stripe.com/docs/webhooks/test
3. Stripe. (n.d.-c). *Webhooks: Verify webhook signatures*. Stripe Docs. https://stripe.com/docs/webhooks/signatures
4. PCI Security Standards Council. (n.d.). *PCI DSS Self-Assessment Questionnaires (SAQ)*. https://www.pcisecuritystandards.org/document_library
5. OWASP. (n.d.-a). *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
6. OWASP. (n.d.-b). *Cross Site Scripting Prevention Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
7. OWASP. (n.d.-c). *REST Security Cheat Sheet* [rate limiting, input validation]. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
8. Internet Engineering Task Force. (2015). *RFC 7519: JSON Web Token (JWT)*. https://www.rfc-editor.org/rfc/rfc7519
9. MongoDB, Inc. (n.d.-a). *MongoDB Manual*. https://www.mongodb.com/docs/manual/
10. MongoDB, Inc. (n.d.-b). *Mongoose ODM Documentation*. https://mongoosejs.com/docs/
11. OpenJS Foundation. (n.d.). *Express — Node.js web application framework*. https://expressjs.com/
12. Meta Platforms, Inc. (n.d.). *React Documentation*. https://react.dev/
13. OpenJS Foundation. (n.d.). *Node.js Documentation*. https://nodejs.org/en/docs
14. Redis Ltd. (n.d.). *Redis Documentation* [including SETNX and key expiry (TTL)]. https://redis.io/docs/latest/
15. Colin McDonnell. (n.d.). *Zod: TypeScript-first schema validation*. https://zod.dev/
16. Vercel. (n.d.). *Turborepo Documentation*. https://turbo.build/repo/docs
17. MUI. (n.d.). *Material UI Documentation*. https://mui.com/material-ui/getting-started/

18. Redux Toolkit. (n.d.). *Redux Toolkit Documentation*. https://redux-toolkit.js.org/
19. TanStack. (n.d.). *TanStack Query (React Query) Documentation*. https://tanstack.com/query/latest
20. Vitest. (n.d.). *Vitest Documentation*. https://vitest.dev/
21. Supertest (ladjs). (n.d.). *Supertest: HTTP assertions made easy* [GitHub repository README]. https://github.com/ladjs/supertest
22. Render. (n.d.). *Render Blueprint (render.yaml) Specification*. https://render.com/docs/blueprint-spec
23. Netlify. (n.d.). *Netlify Configuration File (netlify.toml) Reference*. https://docs.netlify.com/configure-builds/file-based-configuration/
24. bcrypt (npm package, based on Provos, N., & Mazières, D., 1999). *A Future-Adaptable Password Scheme* [original bcrypt algorithm paper, USENIX Annual Technical Conference]. https://www.usenix.org/legacy/events/usenix99/provos.html
25. OWASP. (n.d.-d). *OWASP Top 10:2021*. https://owasp.org/Top10/
26. npm, Inc. (n.d.). *npm-audit: Run a security audit* [CLI command reference]. https://docs.npmjs.com/cli/commands/npm-audit
27. Vite (Evan You / Vite contributors). (n.d.). *Vite: Env Variables and Modes*. https://vitejs.dev/guide/env-and-mode.html
28. Razorpay Software Private Limited. (n.d.). *Razorpay Payment Gateway Integration Documentation*. https://razorpay.com/docs/payments/payment-gateway/
29. PayPal, Inc. (n.d.). *PayPal Checkout Integration Guide*. https://developer.paypal.com/docs/checkout/
30. Mozilla Developer Network. (n.d.). *Set-Cookie: SameSite attribute*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie/SameSite